

Video Voiceover Script

Does Machine Learning Beat the Credit Scorecard? (Trimester 8 Term Project)

François Schmitt (Roll Number: 23035010523)
f.schmitt@op.iitg.ac.in

Reading guide: each block below is read while the corresponding slide is on screen. Times in the headers are targets; the total is about ten minutes. Keep the face visible throughout, per the submission guide. Words in [brackets] are stage directions, not to be read.

Slide 1: Title

(about 35 seconds)

Hello. My name is François Schmitt, I am a student of the B.Sc. Honours Data Science and Artificial Intelligence Online Degree Programme at IIT Guwahati, and this is my Trimester 8 term project. The title is: Does Machine Learning Beat the Credit Scorecard? It is a study in credit risk, on real loan data, and it asks whether the machine-learning advantage that research papers report actually survives the criteria that a real lender cares about. Let me show you what I did and what I found.

Slide 2: The problem

(about 55 seconds)

When a bank decides whether to grant a loan, the decision rests on one number: the probability that the applicant will default. These models are the highest-stakes statistical models in banking, because under the Basel rules and IFRS 9 they determine not just who gets credit, but how much capital the bank must hold. Now, the industry standard is remarkably old technology: a logistic-regression scorecard. Research papers keep reporting that machine learning beats it, but almost all of them measure only one thing, called AUC, which is the ability to rank risky borrowers above safe ones. A lender needs two more things. It needs the probabilities themselves to be right, which is called calibration. And it needs the model to actually make money at its accept-or-reject cutoff. So my question is: does the machine-learning advantage survive all three tests?

Slide 3: Tools and technologies

(about 30 seconds)

The whole project is Python 3.13, running on an ordinary laptop CPU. Pandas and NumPy handle the data, scikit-learn, XGBoost and LightGBM provide the models, and everything lives in a modular package on GitHub, with Jupyter, Git and LaTeX around it. One command reproduces the entire study, all seeds fixed, in about eighteen minutes. Every number I will show you comes from that one command.

Slide 4: Dataset

(about 55 seconds)

The data is the public Lending Club export: six hundred and twenty-one thousand personal loans issued between 2007 and 2015. Two design decisions matter here. First, I only use thirty-six-month loans, so every loan has finished its life before the data ends; no outcome is unknown. Second, and this was the hardest work in the project: the raw export has a hundred and fifty-one columns, and most of them describe what happened after the loan was granted; payments received, recoveries, hardship flags. Any of those trivially predicts the outcome. That is called leakage, and it produces impressive, meaningless results. So I enforced a whitelist of twenty-three fields known on the day of application. Finally, the split follows the calendar: train on 2007 to 2013, calibrate on 2014, test on 2015. Notice that the default rate rises across those vintages. That drift is not a nuisance; it is exactly what a deployed model faces.

Slide 5: Approach and code

(about 55 seconds)

Four models compete, inside one pipeline called creditrisk. The baseline is not a strawman: it is a proper industry-style scorecard. Each feature is binned, each bin gets a weight of evidence, weak features are screened out by information value, and a logistic regression on top produces a score where twenty points double the odds of repayment; the format banks actually deploy. Against it: XGBoost and LightGBM, which are gradient-boosted tree ensembles, and a small neural network. Crucially, all three challengers see exactly the same twenty-three features as the scorecard, through one shared design matrix, so any performance gap comes from the learner, not the preprocessing. Every model is then recalibrated the same way on the 2014 vintage, and the profit analysis uses each test loan's real cash flows: a repaid loan earns its interest, a charged-off loan loses sixty-five percent of principal.

Slide 6: Evaluation

(about 50 seconds)

Before the numbers, the metrics; three questions, each with its own. Discrimination: can the model rank bad borrowers above good ones? AUC is the probability that a random defaulter gets a higher risk score than a random good borrower, and KS is the maximum distance between the two score distributions. Calibration: among loans given, say, ten percent predicted risk, do ten percent actually default? The Brier score and the expected calibration error measure that, and reliability diagrams draw it. And decision quality: realized profit per thousand applicants, across every possible cutoff. Throughout, I use the DeLong test and bootstrap confidence intervals to make sure any gap is real and not luck. The key point: these three questions can rank the models differently. And they do.

Slide 7: Results: discrimination

(about 55 seconds)

First result: the trees win, the neural network loses. XGBoost beats the scorecard by zero point eight five AUC points, LightGBM by zero point seven two, and with two hundred and eighty-three thousand test loans, those gaps are statistically overwhelming; the p-values are astronomically small. But look at the size: not the eight or fifteen point gaps you sometimes read from random-split studies; less than one point. And the neural network is significantly worse than the forty-year-old scorecard. Why so close? Two reasons. The features include the platform's own risk pricing, its grade and interest rate, so every model is partly re-ranking loans that were already ranked once. And the out-of-time split denies the flexible models the flattery of a random split, where training and test share the same economy.

Slide 8: Results: calibration

(about 55 seconds)

Second result: calibration. In these reliability diagrams, a perfectly calibrated model lies on the diagonal. Every grey curve, the raw models, sits above it: they all under-predict risk, because the 2015 test vintage simply defaulted more than the training years. The raw trees are the worst offenders; boosted ensembles optimize ranking, not probabilities. Isotonic recalibration, fitted on 2014, pulls everyone toward the diagonal, but it cannot fix everything, because a correction learned on the past cannot anticipate drift that has not happened yet. That is precisely why regulators demand ongoing monitoring of these models. And here is the nuance I find most interesting: after identical recalibration, the scorecard is still the best-calibrated model of the four. The trees keep the better Brier score, but on pure calibration, the old technology wins.

Slide 9: Results: profit

(about 55 seconds)

Third result: money. These curves show realized profit per thousand applicants as the acceptance cutoff sweeps from strict to lenient. First observation: accepting everyone already earns about seven hundred and five dollars per applicant, because interest income dominates this portfolio. So the model's job is only to prune the riskiest tail. The scorecard's best policy rejects about seven percent of applicants and

gains one point two percent. XGBoost rejects nine percent and gains three point three. The whole battle happens in the zoomed panel on the right; everywhere else the models are interchangeable. The difference between them is about fifteen thousand dollars per thousand applicants, or two point one percent. Modest per applicant, but it scales linearly with volume, which is exactly why large lenders adopt gradient boosting and small ones often, rationally, do not.

Slide 10: Results: interpretability

(about 45 seconds)

What did the models actually learn? Mostly the same thing. On the left, XGBoost's feature importance; on the right, the scorecard's information values. Both agree that the platform's own pricing, the interest rate and the grade, carries the most signal. They disagree mainly in encoding: the scorecard leans on the sub-grade and FICO bins, while XGBoost prefers the continuous interest rate and income; nearly the same information, expressed differently. The rank correlation between the two is zero point five six. The real difference is the interpretability trade-off: the scorecard is a readable sum of points that a loan officer can audit line by line; the ensemble needs post-hoc explanation tools.

Slide 11: Robustness and limitations

(about 50 seconds)

Is this a Lending Club artifact? To check, I repeated the full protocol on an independent dataset, the UCI Taiwan credit-card data. Same ranking: XGBoost beats the scorecard by one and a half AUC points, and the neural network again lands below it. And the honest limitations. The data contain only accepted loans, so every model inherits the platform's own selection; correcting that, reject inference, is an open research problem. The profit model is simple: undiscounted, fixed loss given default. Keeping the platform's grades as features is realistic for an investor but understates what machine learning could add on raw bureau data. And 2015 was a benign year; a recession vintage could reorder the calibration findings.

Slide 12: Wrap-up

(about 55 seconds)

So, does machine learning beat the credit scorecard? A qualified yes. The boosted trees win on all three criteria: discrimination, Brier score after identical recalibration, and profit. The result is statistically unambiguous and it reproduces on independent data. But the margins are modest, only the trees deliver them, the neural network loses outright, and the scorecard remains the best-calibrated model throughout. What did I learn? That leakage control was the highest-leverage work in the entire project. That the model ranking changes with the question you ask, so "best model" is undefined until the deployment question is fixed. And that vintage drift is why these models are monitored, not trusted. Everything you have seen, code, results, and the full report, reproduces with one command from the repository on screen. Thank you for watching.

Total: about 1,450 words; at 145 to 150 words per minute this is 9 to 10 minutes. If a take runs long, the easiest cuts are the final sentences of slides 7 and 9.